Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Основы бизнес-логики и разработка библиотеки для шаблона CoR

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	Архитектура	DevOps		
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репозиторий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	CollabAPI	Машина состояний	Auth	Основы ML	ML в бэкенде	
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- О бизнес-логике
- Паттерн Chain of Responsibility
- Разработка библиотеки
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Понимать что такое бизнес-логика, ее назначение и особенности
2. Пользоваться шаблоном разработки “Цепочка обязанностей”
3. Применять Kotlin DSL для разработки библиотек

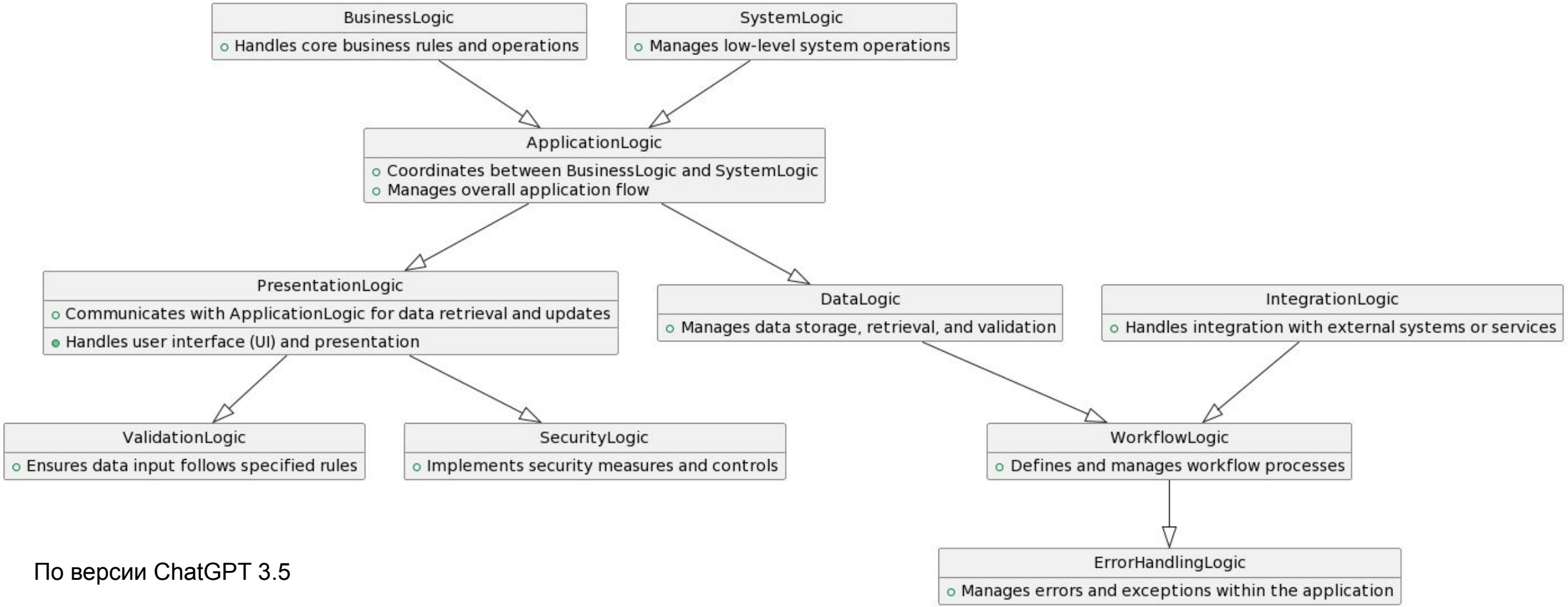
Смысл

Зачем вам это уметь

1. Бизнес-логика - это именно то, ради чего создается все приложение
2. Бизнес-процессы и требования часто меняются, только гибкая логика позволит обеспечить техническое сопровождение проекта за приемлемые ресурсы
3. CoR гораздо гибче чем маркетинговые проекты типа BPMN и BPEL и проще Моделей предметной области

О бизнес-логике

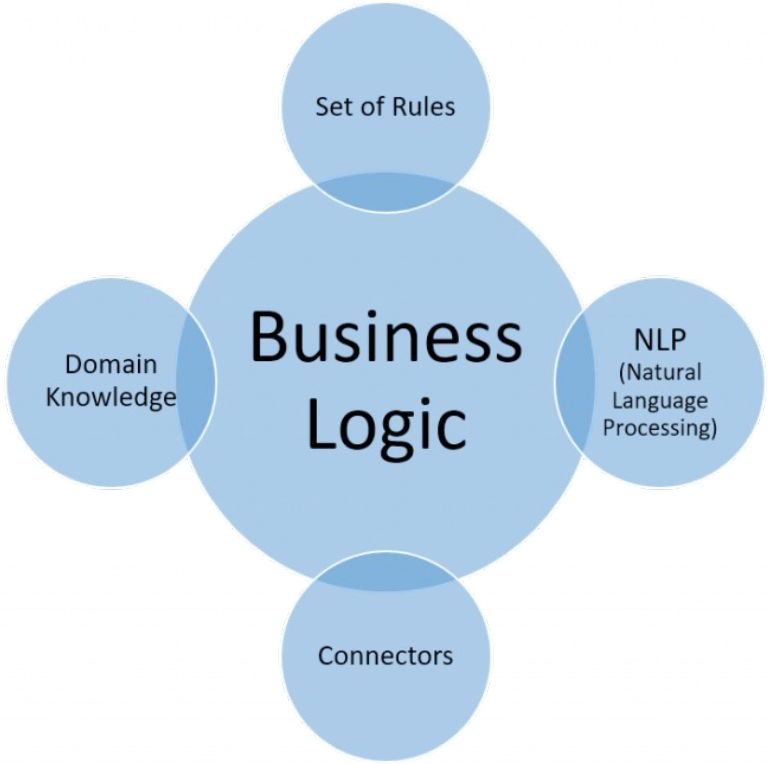
Логика информационной системы



По версии ChatGPT 3.5

Бизнес-логика

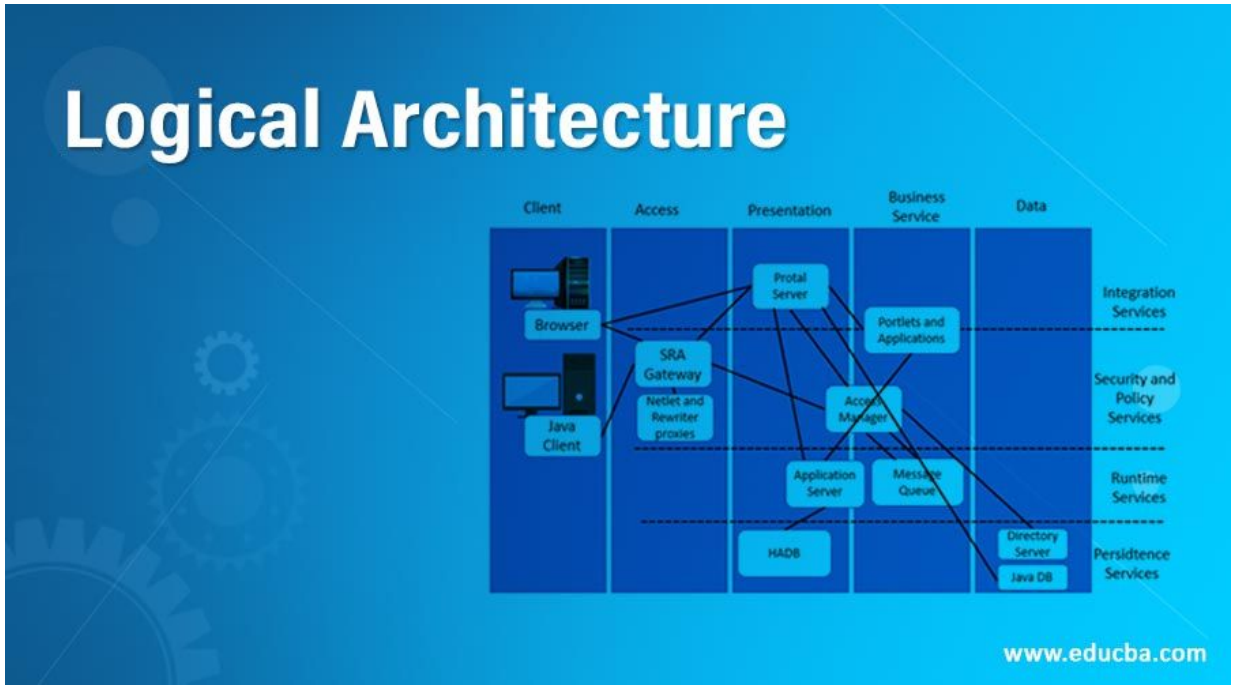
1. Реализация Бизнес-Правил
2. Независимость от Технических Деталей
3. Управление Бизнес-Процессами
4. Доступ к Данным
5. Обеспечение Консистентности
6. Поведенческая Часть Системы
7. Тестируемость и Поддерживаемость
8. Система Знаний Организации



Любые бизнес-требования могут меняться, потому логика должна быть гибкой

Подходы к описанию бизнес-логики

- Транзакционный подход. Реализуем каждую функцию, смотрим что получилось, исправляем ошибки, переделываем для расширения
- Проектирование БД. Представляем что нужно хранить в БД, проектируем ее, обвешиваем обработками
- Проектирование предметной области. Формируем понятия, связи, выстраиваем взаимодействия, реализуем
- Гибкая логика. Исходим из того, что любая функциональность может измениться или расшириться



Chain of Responsibility
и
бизнес-логика на CoR

Вопрос

1. Кто знаком с паттерном Chain of Responsibility?
2. Если знакомы, то опишите Ваш опыт использования паттерна.

2 минуты

Ответы пишем в чат

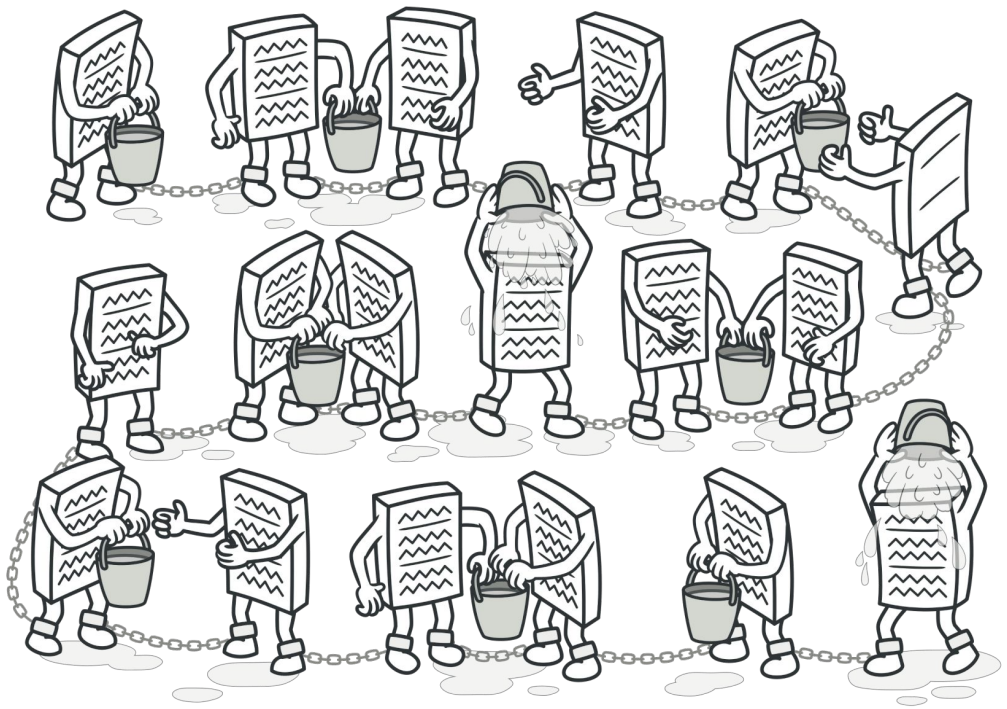
Chain of Responsibility

Шаблон разработки CoR:

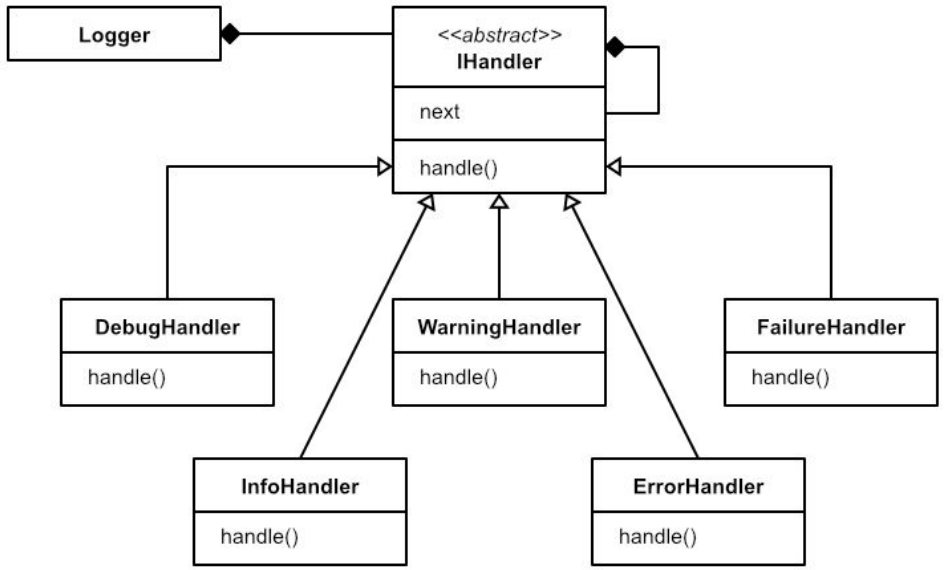
1. Context - DTO для хранения всех возможных состояний текущего бизнес-процесса
2. Handler - выполняет один блок работ и запускает следующий обработчик

На картинке:

1. Контекст - это ведро
2. Обработчик - это костяшка



Классический CoR



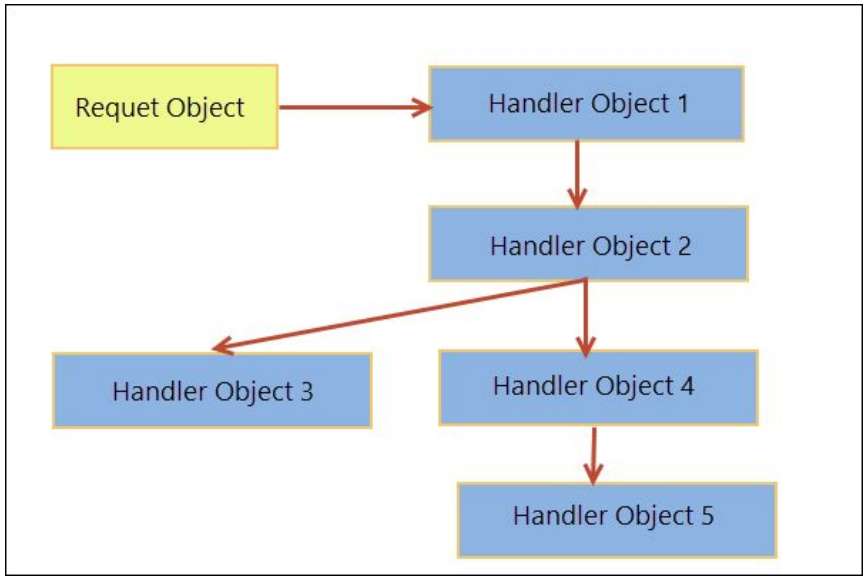
Классический (Книга Банды четырех)

Преимущества:

1. Более гибкая логика

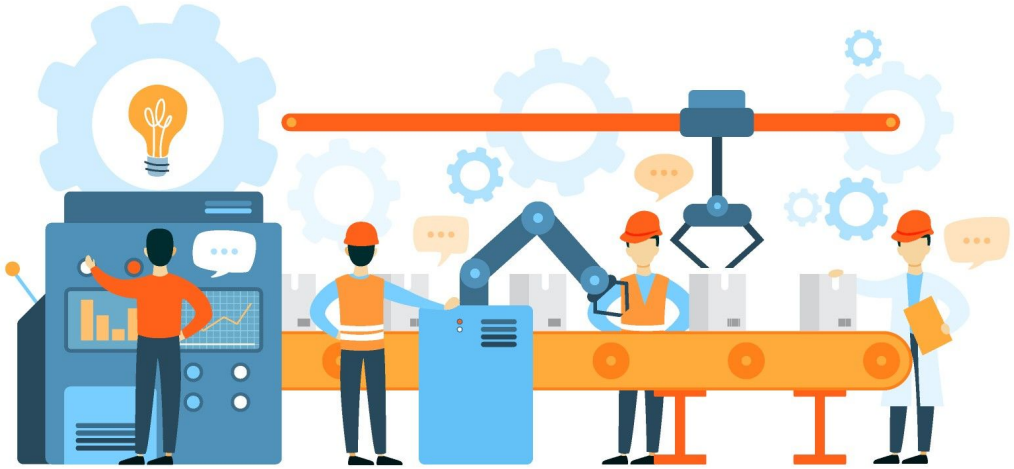
Недостатки:

1. Проблемы вложенности
2. Маршрутизация на уровне обработки
3. Сложнее переиспользовать



Конвейер

1. Context - DTO для хранения всех возможных состояний текущего бизнес-процесса
2. Operation/Handler/Worker - выполняет один блок работ
3. Pipeline/Procesor/Chain - управляет всем бизнес-процессом и запускает обработчики



<u>На картинке</u>:

1. Контекст - это продукт
2. Обработчик - это оператор
3. Процессор - лента конвейера и управляющий конвейером

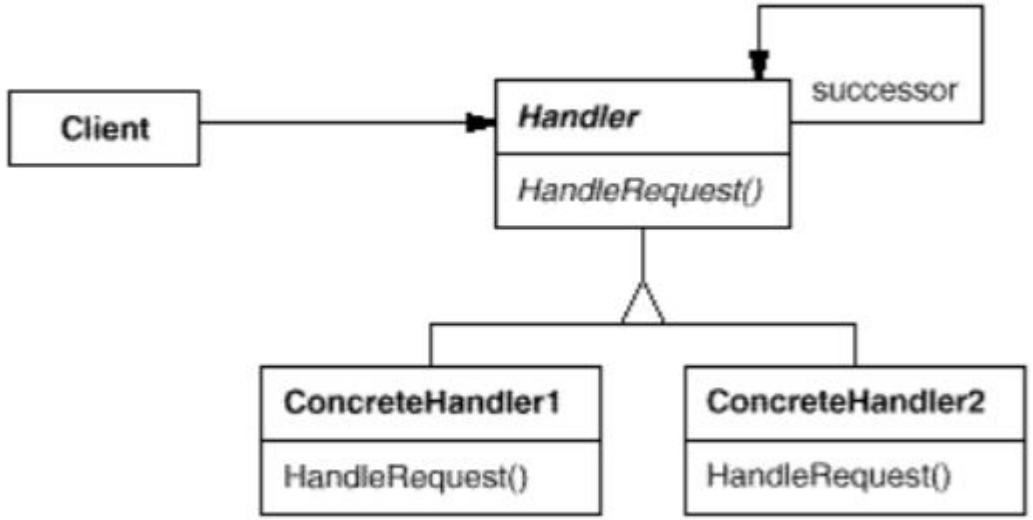
Передача данных обработчикам

```
fun some1(a1: String, a2: String, a3: String, a4: String) {}
fun some2(a1: String, a3: String, a4: String) {}
fun some3(a1: String, a2: String) {}
fun some4(a3: String, a4: String) {}
```

Передача данных обработчикам

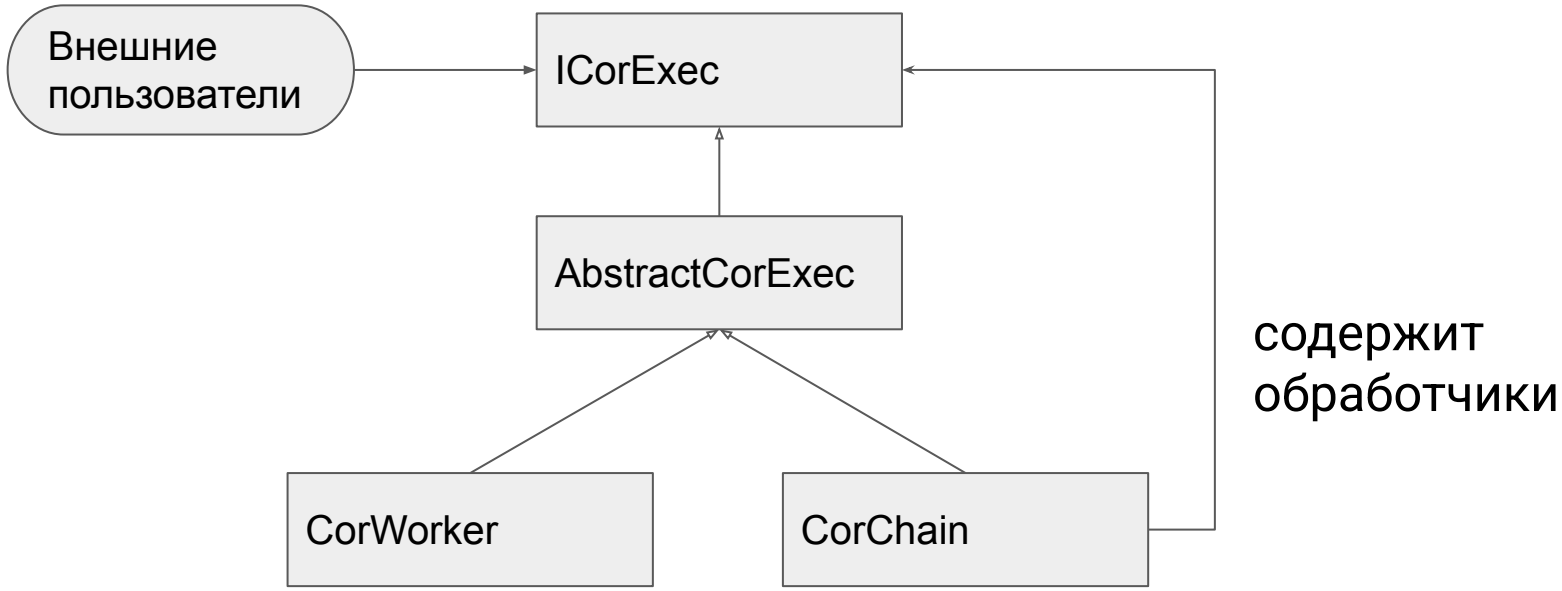
```
fun some1(a1: String, a2: String, a3: String, a4: String) {}
fun some2(a1: String, a3: String, a4: String) {}
fun some3(a1: String, a2: String) {}
fun some4(a3: String, a4: String) {}

// Заменяем на
data class SomeContext(
   var a1: String = “”,
   var a2: String = “”,
   var a3: String = “”,
   var a4: String = “”,
   var a5: String = “”,
)
fun some1(ctx: SomeContext) {}
fun some2(ctx: SomeContext) {}
fun some3(ctx: SomeContext) {}
fun some4(ctx: SomeContext) {}
```



Устройство конвейера

- worker - одинарный обработчик
- chain - цепочка обработчиков, в т.ч. и вложенных chain



Что мы получаем

- Worker и вложенные цепочки - это строительные кирпичики
  - Можно иметь одну единственную корневую цепочку и для разных операций выполнять те или иные worker по условиям
  - Можно иметь разные цепочки на разные операции, хранить их в неком словаре по ключу-операции
- Можно легко добавлять новые worker-ы или менять имеющиеся при изменении бизнес-требований

Требования к контексту

- Является исключительно внутренним классом и не может быть опубликован
- Представляет данные в удобном для внутренней логики виде
- Избегаем наследования во внутренних моделях, минимизируем интерфейсы, используем расширяемость (добавляем необходимые поля)
- Могут использоваться “под-контексты”
- Крайности - на каждую цепочку свой контекст и один контекст на всю систему. Необходимо избегать крайностей

Бизнес-процесс - цепочка Handler’ов

```
/**
 * Блок кода, который обрабатывает контекст. Имеет имя и описание
 */
interface ICorExec<T> {
    val title: String
    val description: String
    fun exec(context: T)
}
```

Внутренняя реализация обработчика

```kotlin
/**
 * Блок кода, который обрабатывает контекст. Имеет имя и описание
 */
interface ICorExec<T> {
    val title: String
    val description: String
    fun exec(context: T)
}
```

```kotlin
/**
 * Реализация ICorExec, выполняется по условию и умеет обрабатывать свои исключения
 */
interface ICorWorker<T> : ICorExec<T> {
    fun on(context: T): Boolean
    fun handle(context: T)
    fun except(context: T, e: Throwable)

    override fun exec(context: T) {
        if (on(context)) {
            try {
                handle(context)
            } catch (e: Throwable) {
                except(context, e)
            }
        }
    }
}
```

Open Source CoR library

https://github.com/crowdproj/kotlin-cor

```kotlin
val chain = rootChain<TestContext> {
    worker {
        title = "Status initialization"
        description = "Check the status initialization at the buziness chain start"

        on { status == CorStatuses.NONE }
        handle { status = CorStatuses.RUNNING }
        except { status = CorStatuses.FAILING }
    }

    chain {
        on { status == CorStatuses.RUNNING }
```

build.gradle.kts

```kotlin
repositories {
    mavenCentral()
}

dependencies {
    val kotlinCorVersion: String by project
    implementation("com.crowdproj:kotlin-cor:$kotlinCorVersion")
}
```

gradle.properties

```
kotlinCorVersion=0.5.5+
```

LIVE

Рефлексия

Вопросы по теме

1. Что такое и зачем нужен паттерн Chain Of Responsibility?
2. Как устроен шаблон Chain Of Responsibility?
3. Что должна и что не должна содержать типичная бизнес-цепочка?

5 минут

Ответы пишем в чат

Тезисы

Подведем итоги

1. COR - это управляющий шаблон для реализации бизнес-логики
2. Обеспечивает однотипность обработчиков и гибкость управления логикой
3. Состоит из контекста, обработчиков и процессоров
4. Типичная бизнес цепочка - валидаторы, трансформация, репозитории
5. Выносить в отдельные классы конкретную реализацию (функциональную логику)

Домашнее задание

1. Сделать модуль бизнес-логики для своего выпускного проекта
2. Реализовать заглушки для запросов с помощью шаблона CoR
3. Выполнить валидацию в модуле бизнес-логики
4. Сделать автотесты для бизнес-логики

Цели вебинара

Проверка достижения целей

1. Понимать что такое бизнес-логика, ее назначение и особенности
2. Пользоваться шаблоном разработки “Цепочка обязанностей”
3. Применять Kotlin DSL для разработки библиотек

Следующий вебинар

Модуль 6: Бизнес-логика проекта

Создание модуля бизнес-логики с использованием шаблона Цепочка Ответственности

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202505-ok-marketplace
- https://refactoring.guru/ru/design-patterns/chain-of-responsibility
- https://ru.wikipedia.org/wiki/Цепочка_обязанностей
- https://www.elegantobjects.org/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары